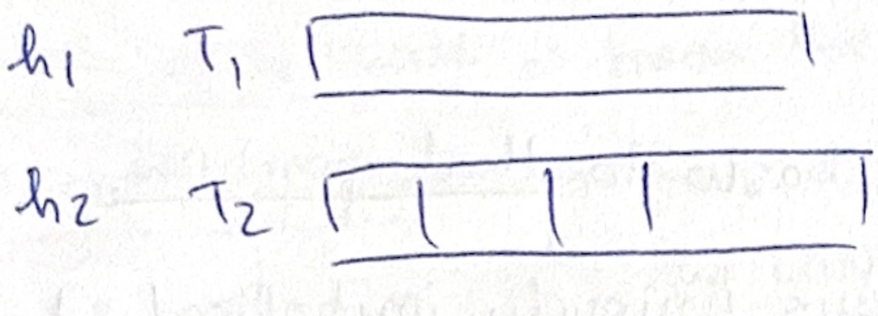
not the "usual" operations / ... but TRANSFORMS

Lecture 11 - DSA

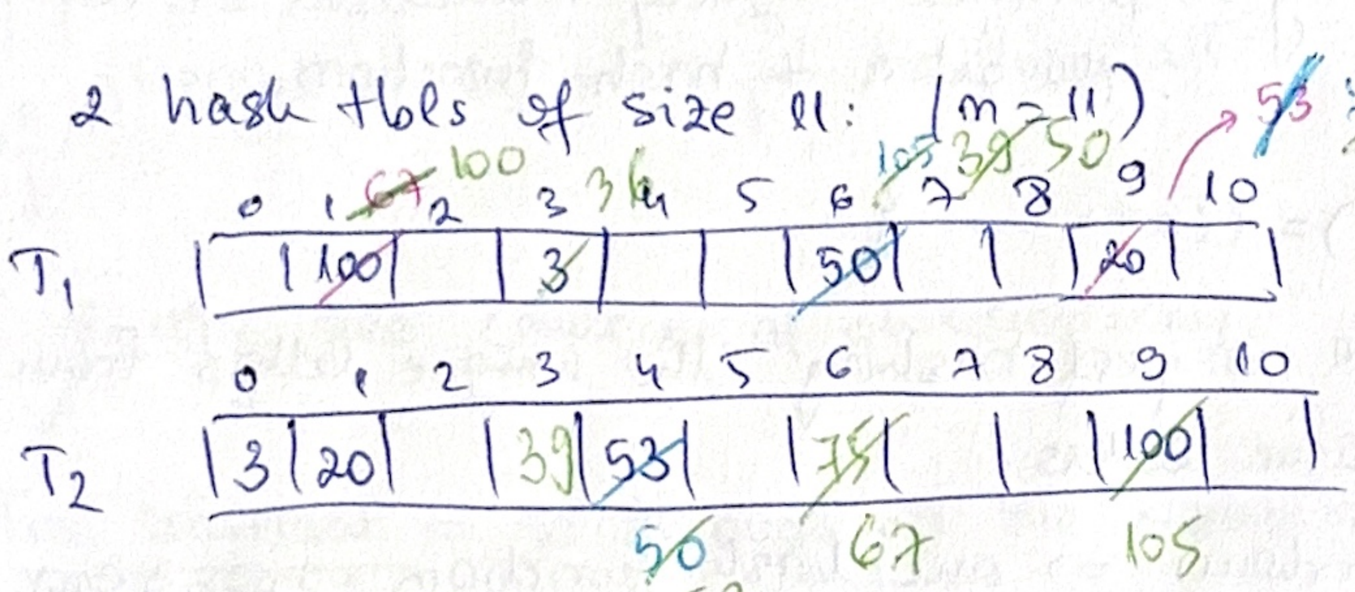
13 May 2026

1) Cuckoo hashing

- 2 hash tables



2 hash tbls of size 11: (m = 11)



$h_1(k) = k \% 11$

$h_2(k) = (k \text{ div } 11) \% 11$

K	h_1	h_2
20	9	1
50	6	4
	$(50 \% 11)$	$(50 / 11) \% 11$
53	9	4

eliminate 20 → put 53 instead. → 20 to T_2

75	9	6	→ 53 to T_2
100	1	9	
67	1	6	

↳ occupied => move 100 to T_2 → put 67 instead.

105	6	9	→ 50 to T_2
3	3	0	→ 53 to T_1

↳ available → 75 to T_2

36	3	3	

↳ occupied => 3 to T_2 on pos 0 (available

39 ← 39 | 6 | 3 | 105 to T_2 on pos

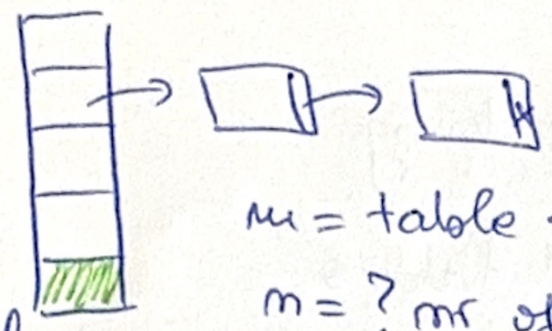
100 to T_1

2) Perfect hashing → no collisions, but you have to know all the elem. that are going to be added in advance. $\Theta(1)$

→ separate chaining $\alpha = \frac{n}{m}$

how much to increase m?

If $m = n^2$ ⇒ 50% chance there are no collisions in the hash tbl.



m = table size

n = ? nr of elements

= hash table of hash tables

↘ only the hash functions that hash to that position.

→ universal hashing → params uniquely initialised s.t. each time you get a ≠ hash function.

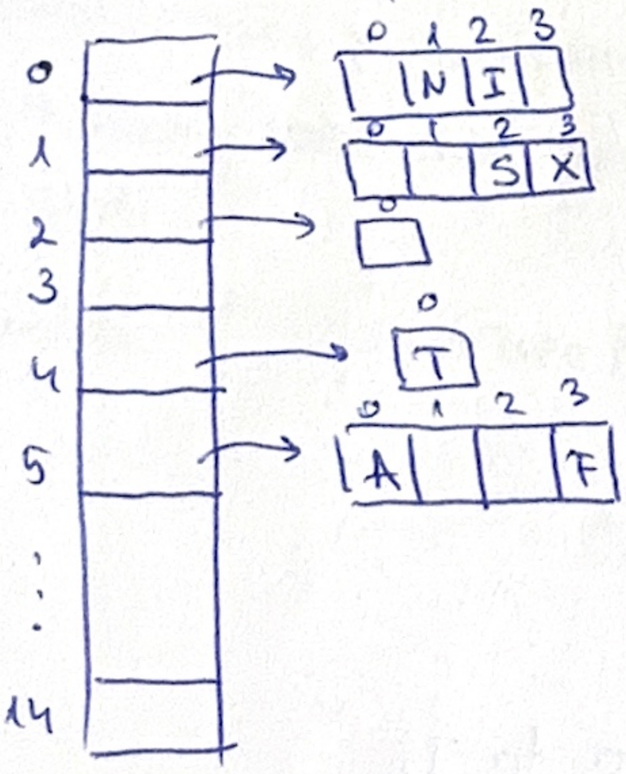
$H = \{H_{a,b}(x) = ((a*x \dots \}$

- Insert in hash table w perf. hashing the ~~unique~~ letters from "A...A".

→ consider only unique letters.

a = 3, b = 2 (random) ⇒ one hash function ⇒ positions for elems

each position may have > letters.



a = 4, b = 11

another hash table whose size is $n^2 = b^2$?

~~a = 5, b = 2~~ a = 2, b = 13

h(K) = 0 only one letter → E.

h(K) = 0

a = 3, b = 5 $((3x+5)\%29)\%4$

F has hash code 6 ⇒ $((3*6+5)\%29)\%4 = 3$ (pos)

A has hash code 1 ⇒ $((3+5)\%29)\%4 = 0$ (pos)

searching = compute hash ~~value~~ table of 3 values. $\Theta(1)$

remove = search + (empty)

insertion = ...

3) Linked hash table - Java

→ separate-chaining based. → don't know the printing order. (depends on size m = table size)

Node:

info: TKey

nextH: ↑Node

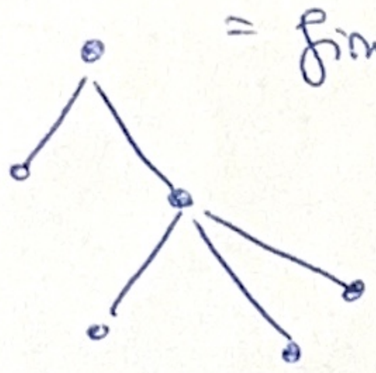
nextL: ↑Node

prevL: ↑Node

20 may 2026

Trees = connected, acyclic graph

= finite set of 0 or more elems = NODES



→ each child is a tree itself.

→ degree node defined by nr. of children

→ leafs (no children)

root

leaf

- **Depth of a node** → from the root, count nr of EDGES. (unique)
- **Height** " — " → from leaf towards one node (not unique)
  ↳ the LONGEST ONE.
- Height of tree = height to the ROOT.

I. K-ary tree (max nr. of children = K)

↙

Tree traversal = go through all the nodes n search

↳ Depth first traversal : start from root → go ↓ from root.

↳ stack

stack : ~~R~~ 2 ~~7 Ø Ø A 7 K 7 P Ø M Ø Y~~

visited : R B T A G J Z K O H Y Q P

→ push from right to left children on the stack

→ if from left to right, still DFS, but start from right side

↳ level order traversal: start from root → go ↓ from root

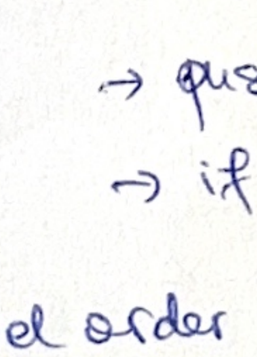
↳ queue

queue: ~~R B G J T A Z K Ø H Ø Ø Y~~

visited: R B G J T A Z K O H Q P Y

II. Binary trees (max 2 children / node)

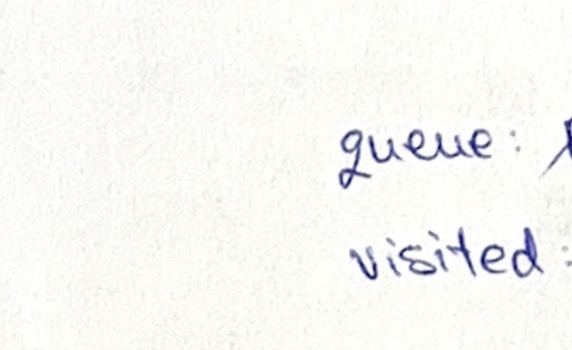
left child, right child → ordered.



= FULL => exactly 2 children for every internal node

(~~COMPLETE~~) → (nr children 0 or 2)

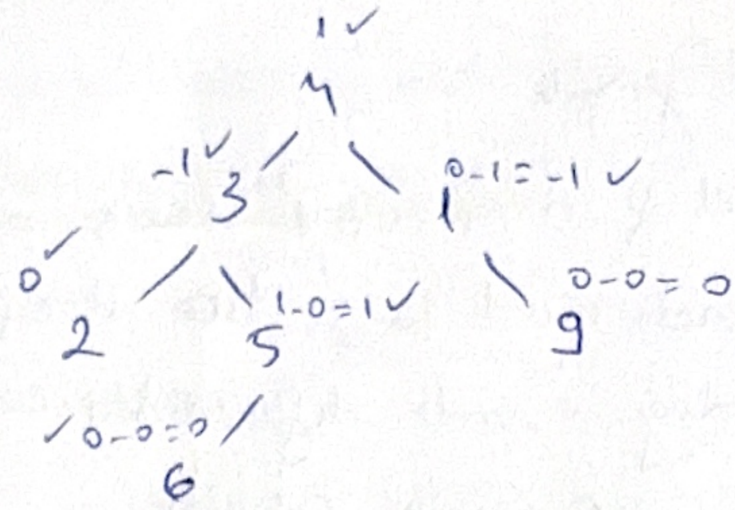
= COMPLETE => can't add any more nodes without increasing the height.



= ALMOST COMPLETE => heap structure

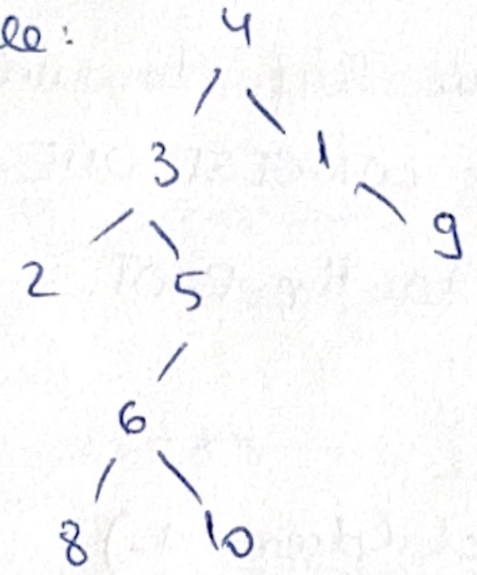
= DEGENERATE => chain

= BALANCED => height(left) - height(right) ≤ 1



• where to add to ~~se~~ keep balanced => near 9.

"random" tree example:



not balanced
not degenerate
not complete
⋮

Properties:
1) n nodes => n-1 edges
2) compleete => $2^{N+1}-1$ nodes = max nr. nodes. 3)
4) mim → N+1 (degenerate)
5) N-1 (compleete)

?

ADT Binary Tree → containes.
• remove - does not make sense
• add

→ create ✓ initTree (bt, left, e, right)